WorldPop

Workshop on Advanced Sampling Methodologies

Simulation and optimization of sampling strategies

Dr. Gianluca Boo, WorldPop, University of Southampton

2025-09-24

Workshop's agenda

- **Day 1**: Introduction to QGIS
- **Day 2**: Automatic update of census EA and national sampling frame
- **Day 3**: Introduction to R programming
- **Day 4**: Introduction to Sampling Design
- **Day 5**: Advanced R programming
- **Day 6**: Earth observation (EO) for sampling design
- **Day 7**: AI and machine learning for sample selection
- **Day 8: Simulation and optimization of sampling strategies**
- **Day 9**: R Shiny for sampling design evaluation
- **Day 10**: Wrap-up and workshop evaluation

Course materials

1. Today's course materials are on **Github**:
   https://github.com/gianlucaboo/sampling_design_workshop
2. Click on **Day 8: Simulation and optimization of sampling strategies**
3. **Copy the url** and **paste it** on https://download-directory.github.io
4. Save the zipped directory on the workshop and unzip it

Today's agenda

- Law of Large Numbers and Central Limit theorem
- Cochran's formula versus Monte Carlo simulation
- Sampling distributions, mean, standard error
- Design comparisons (SRS, stratified, cluster)
- Practical exercises

Sampling design optimization

Field surveys are expensive and require designs that are **accurate** and **efficient**. - A crucial parameter is **sample size**, which is generally computed using **Cochran's formula**.
- Monte Carlo simulations let us evaluate expected performance offers a flexible alternative.

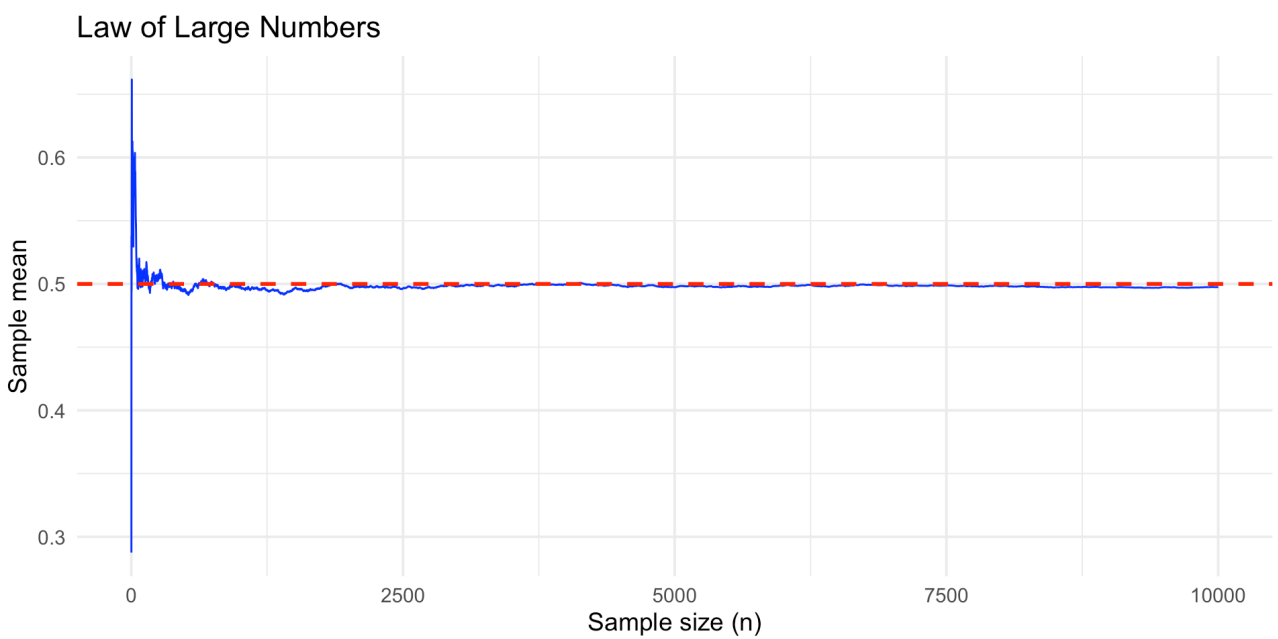
The main **charachteristics of these two approaches** are summarised below:

	Cochran's Formula	**Monte Carlo Simulation**
Purpose	Quick sample size determination under SRS.	Evaluate performance under any sampling design.
Method	Closed statistical formula.	Computational, repeat simulations.
Assumptions	Simple random sampling.	Works with stratified, cluster, PPS, multi-stage designs.
Outputs	Required sample size n.	Bias, variance, RMSE, CI, precision.
Complexity	Simple to apply (pen-and-paper).	Requires coding and computational power.
Accuracy	May fail for complex designs or small populations.	High fidelity as it mimics actual sampling process.

Law of Large Numbers

The **sample mean** ($\bar{X}_n$) of independent, identically distributed (i.i.d.) random variables (X) converges to the **population mean** (μ) as the sample size (n) grows.

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i \quad \longrightarrow \quad \mu \quad \text{as } n \to \infty$$



Law of Large Numbers in R

The **Law of Large Numbers can be demonstrated in R**.

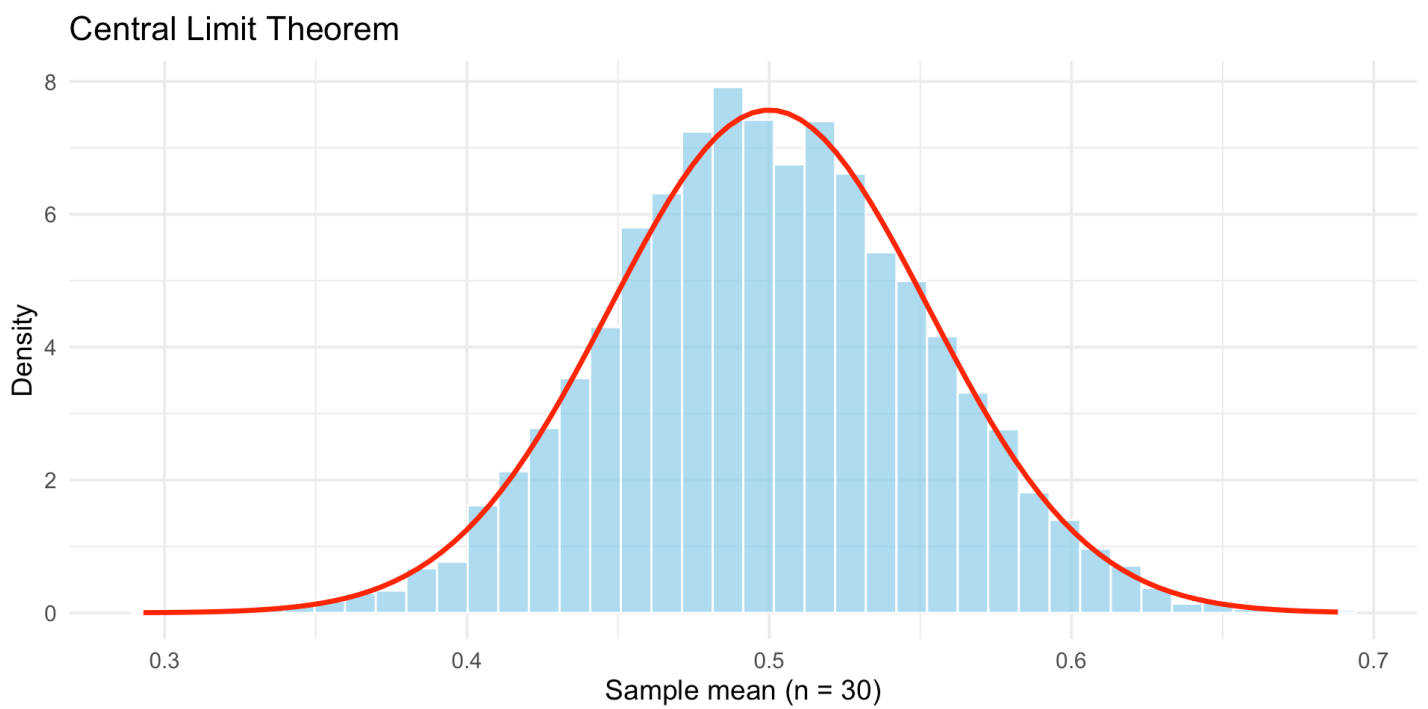
```r
library(tidyverse)
set.seed(123)                        # reproducibility
n <- 10000                           # total number of draws
x <- runif(n, min = 0, max = 1)      # random draws from Uniform(0,1)

true_mean <- 0.5                     # theoretical mean of U(0,1)

# compute cumulative means
cum_means <- cumsum(x) / (1:n)

df <- data.frame(
  n = 1:n,
  cum_mean = cum_means
)

# Plot convergence
ggplot(df, aes(x = n, y = cum_mean)) +
  geom_line(color = "blue") +
```

ℹ️ Try to run the code in your Script Editor.

Central Limit theorem

The distribution of the **sample mean** approaches a **normal distribution** as the sample size (n) increases, regardless of the population distribution.

$$\sqrt{n}\left(\bar{X}_n - \mu\right) \xrightarrow{d} \square(0, \sigma^2)$$



Central Limit theorem in R

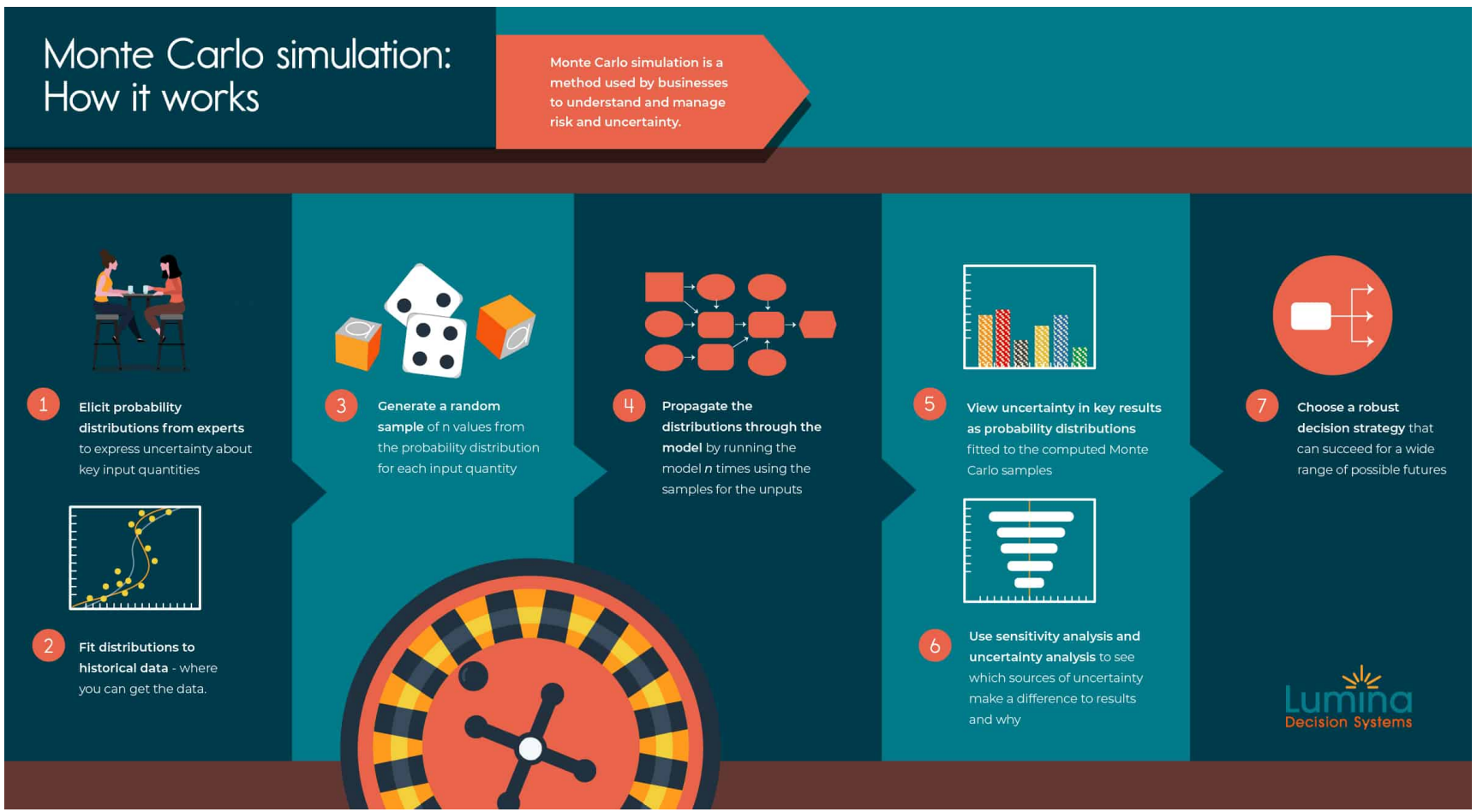
Again, the **Central Limit theorem can be demonstrated in R**.

```r
library(ggplot2)

set.seed(123)

# Parameters
n <- 30               # sample size per experiment
n_sim <- 5000         # number of simulations

# Repeat experiments: sample from Uniform(0,1), compute sample mean
means <- replicate(n_sim, mean(runif(n, 0, 1)))

df_means <- data.frame(sample_mean = means)

true_mean <- 0.5
true_sd <- 1 / sqrt(12)           # SD of U(0,1)
se <- true_sd / sqrt(n)           # standard error of sample mean

# Plot distribution of sample means
```

ℹ️ Try to run the code in your Script Editor.

Monte Carlo simulation

A Monte Carlo simulation involves **repeated random sampling** from a known population model to approximate the **sampling distribution of estimators**.



Monte Carlo simulation workflow (source: Lumina)

Monte Carlo simulation

Monte Carlo simulations can be **evaulated using different metrics** depending on the purpose of the analysis. The most common metrics are:

- **Bias** = average(estimate) - true value
- **Variance** = Var(estimate)
- **RMSE** = sqrt(Bias^2 + Variance)
- **Empirical SE** = sd(estimate)
- **Coverage** of nominal 95% CI = proportion of reps where CI contains true value

ℹ️ These parametric metrics can be complemented by non-parametric statistics, such as the **Empirical Cumlative Distribution function** (ECDF) and the **Kologomorov-Smirnov** statistics (see Boo et al. 2021).

Monte Carlo simulation in R

A Monte Carlo simulation demonstration in R for **simple random sampling** with sample size `n=30`.

```r
library(ggplot2)

set.seed(123)

# Population model: X ~ Normal(μ=5, σ=2)
mu <- 5
sigma <- 2

n <- 30          # sample size
R <- 2000        # number of Monte Carlo replicates

# Monte Carlo loop: compute sample mean each time
means <- replicate(R, mean(rnorm(n, mean = mu, sd = sigma)))

df <- data.frame(estimates = means)

# Plot the distribution of sample mean estimates
ggplot(df, aes(x = estimates)) +
```

ℹ️ Try to run the code in your Script Editor.

Monte Carlo simulation in R

A Monte Carlo simulation for different sample size assessing the **empirical Standard Error of the estimator** for each replicate, assuming that $SE(\bar{\mu}) \approx \sqrt{\text{Var}(\bar{\mu})} = \frac{\sigma}{\sqrt{n}}$.

```r
library(ggplot2); library(dplyr)

set.seed(123)

mu <- 5
sigma <- 2
R <- 2000                              # number of Monte Carlo replicates
sample_sizes <- c(1:100)   # try different n

# Run Monte Carlo for each sample size
results <- lapply(sample_sizes, function(n) {
  means <- replicate(R, mean(rnorm(n, mean = mu, sd = sigma)))
  data.frame(
    n = n,
    mean_est = mean(means),
    se_empirical = sd(means)   # empirical standard error
  )
}) |> bind_rows()
```

ℹ️ Try to run the code in your Script Editor.

Monte Carlo simulation in R

A Monte Carlo simulation for different sample size assessing the **95% CI coverage using** $\text{CI}_{0.95} = \bar{X} \pm z_{0.975} \frac{s}{\sqrt{n}}$ for each replicate, with $\text{CI}_{0.95} \approx 1.96$ for a 95% CI.

```r
library(dplyr); library(ggplot2)

set.seed(123)

mu <- 5
sigma <- 2
R <- 2000
sample_sizes <- 1:100

results <- lapply(sample_sizes, function(n) {
  coverages <- replicate(R, {
    samp <- rnorm(n, mean = mu, sd = sigma)
    xbar <- mean(samp)
    s <- sd(samp)
    se <- s / sqrt(n)

    # 95% CI using normal approximation
    lower <- xbar - 1.96 * se
```

ℹ️ Try to run the code in your Script Editor.

Monte Carlo simulation in R

For **stratified sampling**, the population is divided in **three strata**. Two allocation rules are tested: **proportional** and **Neyman** (stratum size × stratum standard deviation).

```r
library(dplyr); library(purrr); library(tibble)
set.seed(1)                          # Fix random seed for reproducibility
R <- 1000                            # Number of Monte Carlo replicates
N_h <- c(4000,3000,3000)             # Stratum population sizes (3 strata)
H <- length(N_h)                     # Number of strata
N <- sum(N_h)                        # Total population size

# Create artificial stratified population. Each unit belongs to a stratum h
pop <- tibble(h = rep(1:H, times = N_h)) %>%
  mutate(y = case_when(
    h==1 ~ rnorm(n(), 50, 10),   # Stratum 1: mean = 50, sd = 10
    h==2 ~ rnorm(n(), 60, 20),   # Stratum 2: mean = 60, sd = 20
    h==3 ~ rnorm(n(), 55, 12)    # Stratum 3: mean = 55, sd = 12
  ))

true_mean <- mean(pop$y)             # True population mean across all strata

#Define simulation function
```

Stratified weighted sampling

For **stratified weighted sampling**, the population is divided in **three strata** (**proportional** and **Neyman** allocation) and sampled proportionnaly to weigth.

```r
library(dplyr); library(purrr); library(tibble)



set.seed(1)
R <- 1000                       # Number of Monte Carlo replicates
N_h <- c(4000, 3000, 3000)      # Stratum population sizes
H <- length(N_h)
N <- sum(N_h)

# ---- Artificial stratified population ----
pop <- tibble(h = rep(1:H, times = N_h)) %>%
  mutate(y = case_when(
    h == 1 ~ rnorm(n(), 50, 10),
    h == 2 ~ rnorm(n(), 60, 20),
    h == 3 ~ rnorm(n(), 55, 12)
  ),
  # Example of auxiliary weights: could be size, income, etc.
  w_aux = case_when(
```

Monte Carlo simulation in R

It is useful to create **functions replicating different sampling designs**, with **arguments representing the sampling parameters**. These can then be tested through extensive simulations.

```
library(purrr)

res_prop <-
c(1000:1005) %>% # sample size from 200 to 250
map(function(i) {
sim <- simulate_alloc("prop", n=i)

data.frame(
  type = "alloc",
  n=i,
  mean=mean(sim$ybar),
  se=sd(sim$ybar)
)}
) %>%
list_rbind()

# Standard error vs sample size
ggplot(res_prop, aes(x = n, y = se)) +
```

Parallel computation in R

Monte Carlo simulations require **many independent replicates**,each typically running independently. Parallel computation **reduce computation time** significantly using multi-core CPUs or HPC.

```r
library(furrr)

set.seed(123)   # master seed

plan(multisession, workers = 4)  # use 4 cores increase speed ~x4

res_prop <-
c(1000:1005) %>% # sample size from 200 to 250
future_map(function(i) {
sim <- simulate_alloc("prop", n=i)

data.frame(
  type = "alloc",
  n=i,
  mean=mean(sim$ybar),
  se=sd(sim$ybar)
)},
.options = furrr_options(seed = TRUE)
```

Parallel computation in R

Parallel computation should be **used carefully** because a wrong set up can severely impact performance. Key elements to consider are:

Choice of packages

- `parallel` (base R, `mclapply`, `parLapply`)
- `foreach` + `%dopar%` (flexible backends, e.g. `doParallel`)
- `future` + `furrr` (tidyverse-friendly, reproducible)

Number of cores

- Detect with `parallel::detectCores()` or `future::availableCores()`
- Often safer to use *N–1* cores to leave resources for system tasks

Simulation size

- Larger `R` (replicates) benefits more from parallelisation
- Small tasks may not offset communication overhead

Parallel computation in R

Computation time

- Use timing tools to compare sequential vs parallel using the library `tictoc`

Reproducibility

- Always control random number generation
- `set.seed(123)` for sequential
- `.options = furrr_options(seed = TRUE)` for `furrr`

Scalability

- Performance depends on hardware and task complexity.
- Benchmark before running very large simulations

Take home

- **Cochran's formula is quick but limited** as efficient for simple random sampling but less accurate for complex designs.
- **Monte Carlo simulations are flexible** and allow evaluation of any sampling design (stratified, cluster, PPS, multi-stage).
- **Law of Large Numbers and CLT** ensure sample means converge to population means and approximate normality for inference.
- **Optimization is crucial** to balance precision and cost through allocation strategies (e.g., proportional vs. Neyman).
- **Parallel computing accelerates research** but reproducibility, number of cores, and simulation size must be carefully managed.

Resources

- Cochran, W. G. (1977). *Sampling Techniques* (3rd ed.). Wiley.
- Lohr, S. (2019). *Sampling: Design and Analysis* (2nd ed.). CRC Press.
- Boo, G., et al. (2021). A grid-based sample design framework for household surveys. *Gates Open Research*. https://doi.org/10.12688/gatesopenres.13107.1
- Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. *Journal of the American Statistical Association*, 44(247), 335–341. https://doi.org/10.1080/01621459.1949.10483310
- McCallum, A. (2012). *Parallel R: Data Analysis in the Distributed World*. O'Reilly Media.
- Vaughan, D., & Dancho, M. (2024). *furrr: Apply Mapping Functions in Parallel using Futures*. https://furrr.futureverse.org

Exercise

Please download the R script with exercises from GitHub and try to complete it.